Exercise sheet 4[1]

Coding Exercises

Q4.1 Here is the definition of a class for a point (in 3D) whose position is defined by its x, y and z coordinates.

```
#include<cmath>


//===================================
/// Class for a three-dimensional point
//===================================
class Point3D
{

public:

 /// Constructor: Pass x,y and z coordinates
 Point3D(const double& x,
         const double& y,
         const double& z)
    {
     // Store in private member data
     X = x;
     Y = y;
     Z = z;
    }

 /// Function to return the Euclidian distance
 /// to the origin. Const because it doesn't
 /// change the member data
 double distance_to_origin() const
 {
  return std::sqrt(X*X+Y*Y+Z*Z);
 }

private:

 /// x coordinate of point
 double X;

 /// y coordinate of point
 double Y;

 /// z coordinate of point
 double Z;

```

[1]Version 2026/09/28.
Feedback, comments, corrections, applause, etc. to: M.Heil@maths.manchester.ac.uk

2026/09/28, 07:59:00

```
42 };
```

Copy this code into a suitable header file, `point3d.h`, and write a driver code (`point3d.cpp`, say) that includes this header. Check that the class works as intended by creating two `Point3D` objects, one with $x = 3$, $y = 4$, $z = 1$ and one with $x = 12$, $y = 5$, $z = 1$. Use the `distance_to_origin` member function to display the distance of these points from the origin.

Q4.2 In the definition of the `Point3D` class, X, Y and Z are `private` member variables, and so cannot be accessed from outside the class (e.g. in `main`). Once the coordinates of a point are set, they are therefore fixed. Add `public` member functions `void set_x(const double& x)` and `double get_x()` to get and set the values of `x`. Do the same for `y` and `z`. Write some brief code to test these functions. Which of these new functions could (and therefore should!) be declared `const`? Modify your driver code to check that you can actually modify the coordinates of the points after their creation.

Q4.3 Try creating a `Point3D` object without any parameters, i.e. add the following line to your driver code:

```
Point3D c;
```

You will find that this produces a compiler error. Fix the definition of the `Point3D` class to make sure that the above statement works.

Save your code for the Point3D class as we will extend it further next week.

Q4.4 What is wrong with this code?

```
1  class Point3D
2  {
3   public
4     Point3D(double x, double y, double z) : Data(3, 0.0)
5      {
6       Data[1] = x;
7       Data[2] = y;
8       Data[3] = z;
9      }
10
11 private
12     std::vector<double> data;
13     double x() {return Data[0];}
14     double y() {return Data[1];}
15     double z() {return Data[2];}
16
17 }
```

The above code won't compile and even if the syntax errors are fixed, it can't be used with this driver code:

2026/09/28, 07:59:00

```cpp
int main()
 {
  Point2D a(3.0, 4.0, 1.0), b;
  std::cout << a.x << ", " << a.y << ", " << a.z << std::endl;
  std::cout << b.x << ", " << b.y << ", " << b.z << std::endl;

 }
```

Fix both codes!

Q4.5 Here is the definition of two classes:

```cpp
// C++ libraries
#include<iostream>


/// Base class
class Base
{
public:

 /// Pure virtual function
 virtual void pure_virtual_fct() const =0;

 /// Virtual function
 virtual void virtual_fct() const
  {
   std::cout << "Base::virtual_fct()" << std::endl;
  }

 /// Non-virtual function
 void non_virtual_fct() const
  {
   std::cout << "Base::non_virtual_fct()" << std::endl;
  }

};

/// Derived class
class Derived : public Base
{
public:

 /// Implement pure virtual function
 void pure_virtual_fct() const
  {
   std::cout << "Derived::pure_virtual_fct()" << std::endl;
  }

 /// Virtual function
```


2026/09/28, 07:59:00

```
39  void virtual_fct() const
40   {
41    std::cout << "Derived::virtual_fct()" << std::endl;
42   }
43
44  /// Non-virtual function
45  void non_virtual_fct() const
46   {
47    std::cout << "Derived::non_virtual_fct()" << std::endl;
48   }
49
50  /// Function that's only defined in derived class
51  void derived_fct() const
52   {
53    std::cout << "Derived::derived_fct()" << std::endl;
54   }
55
56 };
```

Assume we create two instances of the `Derived` class which we access via a pointers to the `Derived` and `Base` classes, respectively:

```
1  // Make an instance of the derived class; access
2  // via pointer to derived class
3  Derived* derived_pt=new Derived;
4
5  // Make an instance of the derived class; access
6  // via pointer to base class
7  Base* derived_as_base_pt=new Derived;
```

What appears on the screen when you execute this code? (Think about it before writing code to try it out!)

```
1  derived_pt->pure_virtual_fct();
2  derived_pt->virtual_fct();
3  derived_pt->Base::virtual_fct();
4  derived_pt->non_virtual_fct();
5  derived_pt->derived_fct();
6
7   derived_as_base_pt->pure_virtual_fct();
8   derived_as_base_pt->virtual_fct();
9   derived_as_base_pt->non_virtual_fct();
10  dynamic_cast<Derived*>(derived_as_base_pt)->derived_fct();
```


2026/09/28, 07:59:00

Analysis exercise

Analysis Exercise 4: Curves in 3D and their object-oriented representation
Here's some maths for you:

A curve in 3D space can be defined by the three-dimensional position vector

$$\mathbf{r}(\sigma)$$

as a function of a scalar parameter, σ, chosen such that we continue to move along the curve in the same direction as σ increases. For instance σ could (but doesn't have to) be the arclength.
Given the definition of $\mathbf{r}(\sigma)$ for a specific curve, one may wish to plot the shape of the curve in the range $\sigma \in [a, b]$, or compute the arclength

$$\mathcal{L} = \int_{\sigma=a}^{b} |\mathbf{t}| \, d\sigma \tag{1}$$

along that part of the curve. Here

$$\mathbf{t} = \frac{d\mathbf{r}}{d\sigma} \tag{2}$$

is a tangent to the curve. Note that (unless σ is the arclength) this vector is not a unit vector.

The yellow box implies that for every specific 3D line we should be able to:

- obtain the position vector to a point on the line in terms of the parameter σ;
- obtain the tangent vector (2) to the line at a point identified by the parameter σ;
- obtain the arclength (1) along the curve between two different values of σ.

It should therefore also be possible to "plot" the shape of the curve by writing the position vector, evaluated at equally spaced increments in σ, to a file[a].
thus it makes sense to introduce a base class `Line3D` which defines the common interfaces by which we can perform these operations. Here is a first draft for this base class, with comments explaining what we expect each function to do.

```
1
2  /// Draft (!) base class for 3D line
3  class Line3D
4  {
5
```


2026/09/28, 07:59:00

```
 public:

/// Constructor
Line3D();

/// Position vector as a function of the curve parameter
/// sigma: r(sigma)
std::vector<double> position(const double& sigma);

/// Tangent vector as a function of the curve parameter
/// sigma: dr/dsigma
std::vector<double> tangent(const double& sigma);

/// Output n_point points along the curve,
/// at equally spaced values of sigma between sigma=a and
/// sigma=b. filename specifies the name of the file
/// that will contain n_point lines, each containing
/// (only!) the three coordinates (x, y, z) of the point.
/// Coordinates are listed by themselves; no brackets or
/// commas!
void output(const std::string filename&, const double& a,
            double& b, const unsigned & n_point);

/// Return the arclength along the curve between
/// sigma=a and sigma=b.
double arclength(const double& a, const double& b);

 private:

/// Add any additional private data or helper functions
/// that you may need to implement the functions
/// defined above.

[...]


};
```

As an example, consider a specific 3D line, e.g. a helix with constant radius R and pitch $P = 2\pi H$, defined by

$$\mathbf{r}_{\text{helix}}(\sigma) = \begin{pmatrix} R\cos(\sigma) \\ R\sin(\sigma) \\ H\sigma \end{pmatrix}. \tag{3}$$

The C++ representation of this line could be implemented as follows:

```
/// Implementation of the Line3D class for a helix
```


2026/09/28, 07:59:00

```
/// with constant radius and pitch; the latter specified
/// in terms of H = P/(2\pi).
class Helix : public Line3D
{

 public:

  /// Constructor: Pass radius and (measure of)
  /// pitch h = p/(2\pi)
  Helix(const double& radius, const double& h)
  {
    [...]
  }

  [...]
};
```

where `[...]` stands for code that is specific to the `Helix` class and cannot be implemented in generality in the base class.

Tasks:

1. Consider the member functions of the draft `Line3D` base class listed above and check for each function
   - if the function can be implemented in complete generality in the base class. If not, it should be defined as a pure virtual function, implying that it must (and can only) be implemented in a derived class that represents a specific 3D line.
   - if the function should be defined as `const` because its implementation is possible without altering the class's member data (if it has any).
2. The hard distinction between "fully implemented in the base class" and "identified as a pure virtual function", suggested in the previous question is actually too hard. It may be possible (and, in the interest of convenience/generality, desirable) to evaluate certain mathematical operations by suitable numerical approximations. For instance, derivatives can be approximated by finite differences; integrals can be evaluated by the trapezoidal rule, etc.

   Such functions should therefore be defined as virtual functions in the base class and implemented there, using suitable numerical approximations. The (non-lazy) writer of a derived class can then overload the function in their derived class and thus provide the exact, analytical version for their specific 3D line.

   Which of the functions in the draft `Line3D` class listed above fall into that category?

2026/09/28, 07:59:00

3. Use the analysis from the previous two questions to implement the base class `Line3D` in a header file `analysis_exercise4.h`. Define the functions listed in our draft either as virtual, pure virtual or non-virtual. Implement the non-virtual and virtual functions in the `Line3D` class. Make those functions whose implementation shouldn't change the member data `const`.

4. Implement in the same file the class `Helix`, using inheritance from the `Line3D` base class, as described as above. **Note:** Don't bother trying to implement the `Helix::arclength(...)` function but rely on the numerical approximation implemented in the base class. **Hint:** For validation purposes you may use the fact that for $R = 1.37, H = 1.12, a = -0.3$ and $b = 2.2$ you should obtain $\mathcal{L} = 4.42386991219$.

5. When implementing the numerical approximations for the computation of the tangent $\mathbf{t} = \mathrm{d}\mathbf{r}/\mathrm{d}\sigma$ (by finite differencing) and the arclength $\mathcal{L}$ (using the trapezoidal rule), add the protected member data

```
1
2  /// Number of intervals to be used by the trapezoidal rule
3  /// when evaluating the arclength. (Protected, so it can
4  /// be changed by the derived class)
5  unsigned N_interval_trapezoidal;
6
7  /// FD step for the evaluation of the tangent by finite
8  /// differencing (Protected, so it can be changed by the
9  /// derived class)
10 double FD_step_for_tangent;
```

   to the `Line3D` class. Use `N_interval_trapezoidal=100` and `FD_step_for_tangent=1.0e-8` as default settings. This is appropriate for the problems considered here, and given the implementation of these parameters as protected member data, they can be changed in derived classes for which the geometry is so complicated that they require different values.

6. In your report you should

   - briefly summarise the mathematical background and the task
   - briefly explain the appropriate qualifiers for the functions in the `Line3D` base class (const-ness, virtuality).
   - explore how the accuracy of the numerical approximation for the arclength by the trapezoidal rule depends on the number of intervals `Line3D::N_interval_trapezoidal` for the case of the `Helix` class. We would expect that, to obtain a good approximation for $\mathcal{L}$, we have to use a sufficiently large value of

2026/09/28, 07:59:00

`Line3D::N_interval_trapezoidal`. What happens if you set it to 1 (yes: "one"!)? Explain what's going on.

Upload the header file, **`analysis_exercise4.h`** (which should contain the definition of the base class `Line3D` and your implementation of the `Helix` class) and the pdf file containing your results, **`analysis_exercise4.pdf`** to the course's GRADESCOPE page, accessible via Canvas.
It is not necessary to upload the driver code. We will test your header file with our own driver code! Note that for this to work, your header file must not contain the `main()` function!
The pdf file of your report must have **no more than 5 pages**, incl. the title page which must show your student ID.
You can upload your files as many times as you want. The grade for the project will be based on the final version received before the submission deadline:

14:00 on Tuesday of Week 6.

[a] The actual plotting would obviously have to be performed by your favourite plotting package!

2026/09/28, 07:59:00